

Competitive Exam Practice 11 — Answer Key

Parallel Processing and Pipeline Hazards

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

September 5, 2026

Provenance is repeated for auditability. Q1 and Q2 are verified real PYQs from the local GATEOverflow compilation; Q3–Q8 are original GATE-pattern questions. The answers below were independently derived from each stem. The pipeline schedule explicitly marks the load-use bubble and the forwarding path.

Q1 [*GATE CSE 2002 Q2.6, verified real PYQ — GATEOverflow V3 p.81*] **(D)**. Unequal stage delays limit the clock to the slowest stage; data dependencies create data hazards; and shared hardware can create structural hazards. Each can reduce pipelined performance.

Q2 [*GATE CSE 2008 Q76, verified real PYQ — GATEOverflow V3 p.83*] **(A)**. A delayed branch defines a delay slot: the instruction immediately following the branch in program memory executes regardless of whether the branch is taken. The instruction is selected before the branch outcome is known.

Q3 [*Original, GATE-pattern*] **10 cycles, speedup 3.00**. For $k = 5$ stages and $n = 6$ instructions, $T_{\text{pipe}} = k + (n - 1) = 5 + 5 = 10$. The non-pipelined time is $nk = 30$, so the finite-stream speedup is $30/10 = 3.00$.

Q4 [*Original, GATE-pattern*] **cycle 8, 1 bubble**. The schedule is:

Instruction	1	2	3	4	5	6	7	8
LW	IF	ID	EX	MEM	WB			
ADD		IF	ID	stall	EX	MEM	WB	
SUB			IF	stall	ID	EX	MEM	WB

The load value arrives after MEM, so the immediately following ADD needs one bubble even with forwarding. The ADD result is then forwarded to SUB, so no second bubble is needed.

Q5 [*Original, GATE-pattern*] **(A)**. One instruction operates on many data elements, which is SIMD. Independent cores run independent instruction streams on their tiles, which is MIMD.

Q6 [*Original, GATE-pattern*] **(B)**. Two stages demand one unavailable resource at the same time, so this is a structural hazard. Splitting instruction and data memories permits both accesses to proceed; stalling is another valid remedy.

Q7 [*Original, GATE-pattern*] **0.03 CPI**. The expected contribution is

$$\Delta \text{CPI} = f_{br} p_{miss} P_{flush} = 0.20 \times 0.05 \times 3 = 0.03.$$

Q8 [*Original, GATE-pattern*] **(A) and (C)**. Forwarding removes many ALU-to-ALU RAW stalls, but a load-use value can arrive too late and still requires a bubble. A misprediction invalidates younger wrong-path instructions. Register renaming removes false WAR/WAW name conflicts, not a true RAW dependence on a value that has not been produced.